

# HW 2B — Web Security with OWASP Juice Shop

Secure feature design · front-end login form · self-exploitation

**Repository:** <https://github.com/cmangalwedhe/csce-477-hw2>

## Part 1 — Secure Feature Design

---

Testing OWASP Juice Shop, I confirmed three vulnerabilities. **(1) SQL Injection login bypass:** entering `' OR 1=1--` as the email with any password logs you in as admin, because the query concatenates input directly. **(2) Cross-Site Scripting:** the search bar reflects the `q` parameter unescaped, so `<iframe src="javascript:alert(1)">` executes. **(3) Broken authentication:** weak passwords and no rate limiting allow brute force. Mitigations: use **parameterized queries** (input becomes data, never SQL), **output encoding / CSP** (markup renders as text), and **bcrypt hashing + rate limiting + lockout**. These remove the attacker's ability to alter query structure, inject script, or guess credentials at scale.

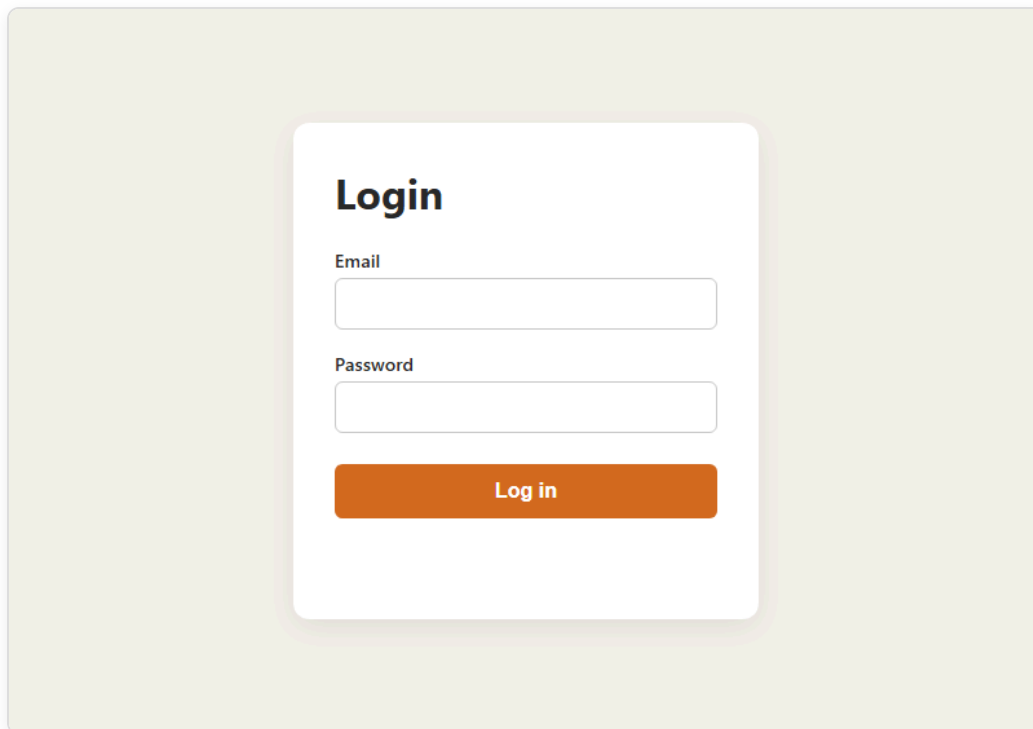
### Secure password handling example

```
const bcrypt = require("bcryptjs");
// On registration - store only the salted hash (cost factor 12):
const hash = bcrypt.hashSync(plaintextPassword, 12);
// On login - compare without ever storing or logging plaintext:
const ok = bcrypt.compareSync(submittedPassword, hash);
```

## Part 2 — Front-End Form Implementation

---

I built a login form ( `public/index.html` + `app.js` ) styled to resemble Juice Shop, backed by a small Express server ( `server.js` ). The **client** blocks empty submissions and checks that the email contains `@` and the password is  $\geq 8$  characters before sending. The **server** re-runs identical validation (clients can be bypassed), then authenticates against an in-memory SQLite DB using **parameterized queries**, **bcrypt**-hashed passwords, **generic error messages** (prevents account enumeration), and a **rate limiter** (10 attempts/15 min). Status text is written with `textContent`, never `innerHTML`, so echoed input can't execute. Run with `npm install && npm start`, then open `http://localhost:3000`.

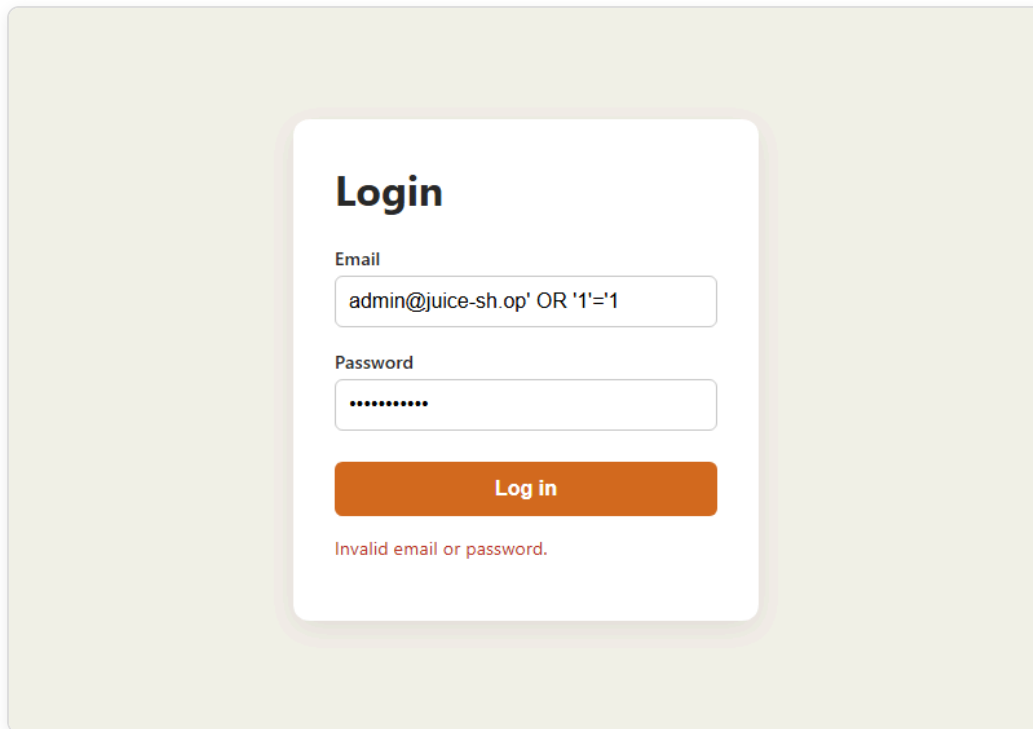


The login form running at localhost:3000.

## Part 3 — Exploiting My Own Form

**Steps taken.** I attacked my own login with (a) the SQL-injection auth-bypass `admin@juice-sh.op' OR '1'='1`, (b) a raw `' OR 1=1--`, and (c) an XSS payload `<script>alert(1)</script>@x` in the email field — via both the browser UI and `curl`.

**Result — attacks failed.** Every payload returned `Invalid email or password`. No login, no script execution. The parameterized query binds the payload as a literal string that matches no row; the `textContent` sink renders markup as inert text. A separate demo (`vulnerable-demo.js`) proves the *same* payload succeeds against concatenated SQL, returning the admin row — confirming the fix is what matters.



SQL-injection payload in the email field — rejected with a generic error.

**Fix.** If the form *were* vulnerable, the fix is the one already applied: **parameterized / prepared statements** for every query (plus output encoding for any reflected value). Never build SQL by string concatenation.

```
=== SQL injection auth-bypass attempt ===
{"success":false,"message":"Email must contain '@'."}
=== injection with valid-looking email ('admin@juice-sh.op' OR '1'='1') ===
{"success":false,"message":"Invalid email or password."}
=== XSS payload in email field ===
{"success":false,"message":"Invalid email or password."}
=== correct credentials ===
{"success":true,"message":"Login successful."}

=== vulnerable-demo.js (concatenated SQL, SAME payload) ===
```

```
Executed query: SELECT * FROM users WHERE email = 'admin@juice-sh.op' OR '1'='1'  
Leaked row: { id: 1, email: 'admin@juice-sh.op', password_hash: 'real-hash' }  
Rows returned: 1 → AUTH BYPASS SUCCEEDED  
Parameterized version with SAME payload → matched NO rows (injection neutralized).
```